

Programación Dinámica Top Down

Técnicas de Diseño de Algoritmos, UBA. Compilado: 30/09/2026.



DEPARTAMENTO
DE COMPUTACION
Facultad de Ciencias Exactas y Naturales - UBA

Contenidos

Programación dinámica	1
Principio de optimalidad	1
Memoización	1
Poniendo todo junto	1
Cómo resolver problemas con PD	2
Ejercicios	3
El ejército del rey	3
Contar caminos en DAGs	4
OptiPago	6
Las legiones del César	8

| Programación dinámica

Primero aclaremos algo, la palabra **programación** en programación dinámica (al igual que en programación lineal/entera) viene de la idea de un itinerario/tomar una seguidilla de decisiones, el objetivo era tratar de optimizar algún problema con múltiples decisiones.

En nuestro caso, las decisiones van a ser subestados de un problema recursivo.

| Principio de optimalidad

Definición: Dado un problema de optimización decimos que este cumple el principio de optimalidad si dado un estado del problema el valor óptimo del mismo se puede conseguir usando el valor óptimo de sus subestados que usa.[†]

Esto es importante, para aplicar PD queremos que nuestros problemas cumplan el principio de optimalidad ya que nos permite guardar el valor de subproblemas ya computados para reusarlos sin volver a calcularlos sin perder optimalidad.

| Memoización

Recordemos la idea, memoizar nos ayuda guardando el cómputo de estados que se repiten mucho cuando exploramos el árbol de backtracking, mientras más solapamiento haya de subestados llamados más vamos a podar el árbol de recursión.

| Poniendo todo junto

En nuestro caso podemos ir de un algoritmo de backtracking a uno de PD simplemente almacenando cada estado en un diccionario como muestra el pseudocódigo que sigue, donde siguientes(S) nos

[†]Principle of Optimality: An optimal policy has the property that whatever the initial state and initial decision are, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision.

da el conjunto de siguientes subestados a evaluar, combinar combina los resultados de los llamados recursivos (por ejemplo una suma, min, max, $\vee$, $\wedge$, etc):

M: Estados $\rightarrow$ Valor // diccionario de estados, dado un estado devuelve el
// cómputo de la recursión si ya fue calculado.

```
BT(S) {  
  if CASO_BASE(S) {  
    M[S] = B  
    return M[S]  
  }  
  
  if M[S] == UNDEF {  
    temp = valor_inicial  
    for S' in siguientes(S) {  
      temp = combinar(temp, BT(S'))  
    }  
    M[S] = temp  
  }  
  
  return M[S]  
}
```

Desglosemos un poco esto, fuera de un caso base lo que hacemos es fijarnos si ya computamos el estado, si lo hicimos devolvemos $M[S]$, sino iteramos sobre el conjunto de siguientes estados posibles de una función recursiva como hacíamos en backtracking y combinamos el resultado de las llamadas recursivas, igual que hacíamos en backtracking, solo que al final nos guardamos el valor computado en $M[S]$ y devolvemos eso.

¿Cuál es la complejidad de esto? Recordemos que en backtracking la complejidad en tiempo era

$$O(\text{llamados recursivos} \times \text{costo adentro de cada llamado})$$

Ahora veamos qué pasa con el esquema anterior, podemos pasar por un estado muchas veces pero solo lo computamos una única vez, entonces tenemos una cota superior de $O(|S|)$ que vamos a calcular en su totalidad una sola vez y luego si se vuelve a llamar a ese estado es simplemente una operación en $O(1)$. Juntando todo esto tenemos que la complejidad en PD es

$$O\left(\sum_{S \in \text{estados}} \text{costo}(S)\right)$$

! Cómo resolver problemas con PD

Dado un problema que queremos resolver con PD podemos aplicar el siguiente esquema:

- Identificar cuáles son los estados posibles de nuestro problema.
- Escribir la función recursiva (como si hiciéramos backtracking).[‡]
- Demostrar/Observar que hay superposición de problemas.
- Memoizar los resultados para no tener que recalcularlos.

[‡]A veces se puede partir del segundo paso aplicando el algoritmo más ingenuo de BT que se pueda y luego observando qué estados hay, en este caso puede haber que refinar un poco la recursión hasta llegar a lo que se busca.

Ejercicios

El ejército del rey

Problema

El rey Cambyse está interesado en armar ejércitos en una serie de días consecutivos. Más aún, le interesa que el número de personas de su ejército en el día $i \in \mathbb{N}$ sea equivalente a la suma del número de personas del ejército que formó el día $i - 1$ e $i - 2$. La excepción para esto es en los días 0 y 1, en los cuales su ejército tendrá 1 sola persona. Para él es muy complicado determinar este número, así que nos pidió que lo ayudemos. Dado un día $n \in \mathbb{N}$, tenemos que devolver el número de personas de su ejército.

- Escribir la función recursiva que quiere calcular el rey en términos de $n \in \mathbb{N}$. ¿Es posible que esta función devuelva distintos valores para el mismo n ?
- ¿Qué complejidad tiene esta función si se la implementa de forma directa?
- Dibujar el árbol de llamadas recursivas para el día 4. ¿Se calculan las mismas cosas más de una vez? ¿Cuáles?
- Implementar un algoritmo recursivo en el lenguaje que prefieran que evite calcular dos veces lo mismo, guardando los resultados ya calculados para utilizarlos posteriormente.

Ayuda: Considerar guardar los valores ya calculados en un arreglo de n posiciones, y acceder a ese arreglo para ver si el valor actual ya fue calculado o no.

- Determinar la complejidad temporal y espacial de este nuevo algoritmo. ¿Cómo se compara con el algoritmo que usa solo backtracking?
- Optimizar el algoritmo anterior para utilizar una complejidad espacial $O(1)$.

Solución:

- El problema nos dice que para los días 0 y 1 tenemos 1 sola persona y para cualquier otro día la cantidad de personas es en base a la suma de los dos anteriores de lo cual podemos construir la siguiente función recursiva:

$$f(k) = \begin{cases} 1 & \text{si } k = 0 \vee k = 1 \\ f(k-1) + f(k-2) & \text{si } k \geq 2 \end{cases}$$

- El árbol de recursión de $f(k)$ tiene como mucho altura k que viene de las recursiones sobre $f(k-1)$ en el segundo caso entonces podemos sobre acotar asumiendo que es completo y obtener un árbol binario de altura k , este tiene $2^k - 1$ nodos que son nuestros estados. Además cada estado hace trabajo $O(1)$ con lo cual podemos acotar la complejidad[§] por

$$O(2^k)$$

- Usemos el framework que detallamos antes, ya tenemos la función recursiva y nuestros estados son los posibles días.

```
M: N -> Z, array de enteros de tamaño n + 1 inicializados todos en -1,
M[0] = M[1] = 1
f(k) {
    if k == 0 or k == 1 {
```

[§]Se podría hacer un análisis más fino que en realidad nos da $O(\phi^n)$ donde ϕ es el número áureo.

```

    return 1
}

if M[k] == - 1 {
    l = f(k - 1)
    r = f(k - 2)
    M[k] = l + r
}

return M[k]
}

```

- d) La complejidad temporal es la cantidad de estados posibles por el costo en cada uno de sus estados, en este caso hay n estados posibles si llamamos a $f(n)$ y en cada estado se hace trabajo $O(1)$ con lo cual nos queda una complejidad lineal en $O(n)$.
- e) Tarea para PD bottom up.
- f) Tarea para PD bottom up.

Contar caminos en DAGs

Problema

- a) Proponer un algoritmo que tome tiempo $O(n + m)$ para contar la cantidad de caminos distintos en un DAG entre dos vértices s y t .
Ayuda: Considerar que su función recursiva calcule la cantidad de caminos distintos que terminan en un vértice dado del DAG.
- b) Demostrar su correctitud y complejidades temporal y espacial.

Solución:

- a) Nos dan una ayuda que es dar una función recursiva que calcule cuantos caminos terminan en un vértice v .

Expandamos un poco esta idea para grafos en general, si estamos en un vértice v y queremos contar los posibles caminos de s a v podemos pensar en los caminos de s que llegan a sus antecesoros y luego extenderlos con v . El problema es que podríamos tener ciclos, pero estamos en un DAG, así que esos caminos van a estar bien formados, es decir van a ser simples y acíclicos. Esto nos da una forma de contar los posibles caminos de s a v usando sus antecesoros recursivamente, tenemos la siguiente recursión:

$$f(v) = \begin{cases} 1 & \text{si } v = s \\ \sum_{w \rightarrow v} f(w) & \text{si no} \end{cases}$$

La semántica de esta función es: la cantidad de caminos que hay de s a v . Notar que la suma devuelve 0 si no tiene aristas entrantes y es distinto de s .

Notar que como el grafo es acíclico esta función siempre termina. Ahora la solución al problema original se consigue llamando a $f(t)$.

Escribamos el código de un algoritmo con PD que use esta función recursiva:

```

// Inicialización
M = [UNDEF] * n // Array de tamaño n inicializado
                // en -1 (nuestro `UNDEF`)
N_in // Listas de adyacencia de antecesores

f(v) {
  if v == s {
    return 1
  }

  if M[v] == UNDEF {
    sum_paths = 0
    for w in N_in(v) {
      sum_paths += f(w)
    }
    M[v] = sum_paths
  }

  return M[v]
}

```

- b) **Complejidad:** ¿Cuántos estados posibles hay? Hay tantos estados como vértices en el grafo, es decir $|V|$. Además en cada estado se puede hacer a lo sumo $n - 1$ trabajo que es la cantidad máxima de potenciales aristas entrantes a cada vértice, usando esto un análisis naive nos diría que es $O(n^2)$ pero en realidad es un poco mejor.

Veamos el peor caso, el de evaluar a todos los antecesores para cualquier vértice, en tal caso tenemos que el costo total está acotado por

$$\sum_{v \in V} |N^-(v)| = m$$

Pero además tenemos el costo de inicializar el array de memoización, esto considera el caso de que el DAG no sea conexo, juntando tenemos que la complejidad final es de

$$O(m + n)$$

Correctitud: Como estamos en DAGs podemos usar que existe un orden topológico. Dado alguno de estos órdenes $\{v_1, \dots, v_n\}$ vamos a hacer inducción fuerte sobre el índice en el orden topológico y probar que vale el siguiente predicado

$$P(k) : f(v_k) = \#\{\text{Caminos de } s \text{ a } v_k \text{ en el orden topológico}\}$$

Caso base: $k = 1$ tenemos dos casos:

- $v_1 = s$ caemos en el primer caso de nuestra recursión que devuelve 1 y es lo que esperábamos ya que en un DAG hay un único camino de s a s que es el camino trivial de un vértice.
- $v_1 \neq s$ entonces $v_1 < s$ en el orden topológico y por lo tanto no hay caminos de s a v_1 , caemos en el segundo caso de nuestra recursión pero como v_1 es el primero del orden no tiene aristas entrantes y por lo tanto la suma evalúa a 0.

Caso inductivo: Vamos a hacer inducción fuerte, supongamos que probamos $P(j)$ para todo $1 \leq j \leq k$ y queremos ver que vale $P(k + 1)$. Supongamos que $v_{k+1} \neq s$, si fuesen iguales es el mismo argumento que el caso base. Ahora podemos separar v_{k+1} en dos casos más

- Si v_{k+1} no tiene antecesores devolvemos 0 ya que la recursión nunca evalúa.

- Si v_{k+1} tiene antecesores entonces están antes en el orden topológico, si existe un camino de s a v_{k+1} como es un DAG tiene que pasar por algún vértice que está antes en su orden topológico. Como w está antes entonces es igual a algún v_j en el orden que fijamos con algún $1 \leq j \leq k$ entonces por la hipótesis inductiva sabemos que $P(j)$ vale y por lo tanto

$$f(v_j) = f(w) = \#\{\text{Caminos de } s \text{ a } w \text{ en el orden topológico}\}$$

Ahora cada uno de estos caminos a sus antecesores es extendible a un camino de s a v_{k+1} ya que el grafo es acíclico nunca visitamos a v_{k+1} antes. Además los caminos que llegan a antecesores son distintos por lo cual si sumamos sobre todos los caminos de los antecesores de v_{k+1} conseguimos la cantidad total de caminos que hay de s a v_{k+1} . ■

OptiPago

Problema

Tenemos un multiconjunto $B = \{b_1, \dots, b_n\}$ de valores de billetes y queremos comprar un producto de costo c en una máquina que no da vuelto. Para adquirirlo debemos cubrir su costo usando un subconjunto de nuestros billetes. Queremos pagar con el mínimo exceso posible y, entre las opciones con igual exceso, usar la menor cantidad de billetes. Por ejemplo, si $c = 14$ y $B = \{2, 3, 5, 10, 20, 20\}$, conviene pagar 15 con los billetes de 10 y 5.

En cada decisión podemos usar el billete b_n o no usarlo.

- Definir las soluciones candidatas y válidas, y expresar la función objetivo.
- Definir las soluciones parciales y una función recursiva $\text{OptiPago}(B, c)$. Indicar la llamada inicial y demostrar su correctitud.
- Implementar la función recursiva y analizar sus complejidades.
- Reescribirla como $\text{OptiPago}_{B(i,j)}$, dejando fijo B . Comparar la complejidad con y sin memoización.
- Adaptar el algoritmo para memoizar los estados.

Solución:

- Para cada billete decidimos si lo usamos o no, así que una solución es un vector binario $X = (x_1, \dots, x_n)$. Las soluciones candidatas y válidas son

$$\text{Sols} = \{0, 1\}^n, \quad \text{Sols_validas} = \left\{ X \in \text{Sols} \mid \sum_{i=1}^n x_i b_i \geq c \right\}.$$

Comparamos las soluciones lexicográficamente: primero el monto pagado y, en caso de empate, la cantidad de billetes. La función objetivo es

$$\min_{X \in \text{Sols_validas}} \left(\sum_{i=1}^n x_i b_i, \sum_{i=1}^n x_i \right).$$

- Una solución parcial es una secuencia de decisiones sobre un prefijo de los billetes: $\text{Sols_parciales} = \cup_{k=0}^n \{0, 1\}^k$. Para construir la recurrencia miramos el último billete

disponible, b_n . Toda solución no vacía cae en dos casos: no usarlo, o usarlo. En el segundo caso queda por cubrir $j - b_n$ y luego sumamos $(b_n, 1)$ a la solución obtenida.

Definimos $\text{OptiPago}(B, j) = (c', q)$ como la mejor forma de cubrir el monto restante j usando los billetes de B . Si no existe, devolvemos (∞, ∞) :

$$\text{OptiPago}(B, j) = \begin{cases} (0, 0) & \text{si } j \leq 0 \\ (\infty, \infty) & \text{si } j > 0 \text{ y } B = \emptyset \\ \min_{\text{lex}}(\text{OptiPago}(B', j), \text{OptiPago}(B', j - b_n) + (b_n, 1)) & \text{en otro caso.} \end{cases}$$

donde $B' = B \text{ setminus } \{b_n\}$. La llamada inicial es $\text{OptiPago}(B, c)$.

Correctitud. Probamos por inducción en $n = |B|$ que, para todo j , la función devuelve la tupla óptima. Si $B = \emptyset$, para $j \leq 0$ la respuesta es $(0, 0)$ y para $j > 0$ es (∞, ∞) . Para el paso inductivo, las soluciones se separan entre las que no usan b_n y las que sí lo usan. Por hipótesis inductiva, las dos llamadas devuelven el óptimo de su clase; tomar el mínimo lexicográfico devuelve el óptimo global.

c) La implementación directa es:

```
def optipago(B, i, j):
    if j <= 0:
        return (0, 0)
    if i == 0:
        return (float("inf"), float("inf"))

    sin = optipago(B, i - 1, j)
    monto, cantidad = optipago(B, i - 1, j - B[i - 1])
    con = (monto + B[i - 1], cantidad + 1)
    return min(sin, con)
```

En el peor caso hay dos llamados por billete: el tiempo es $O(2^n)$ y la pila ocupa $O(n)$ espacio.

d) Dejando fijo B , el estado es (i, j) : i indica cuántos billetes del prefijo están disponibles y j cuánto falta cubrir. Llamados con el mismo estado tienen el mismo subproblema, y por eso se pueden memoizar:

$$\text{OptiPago}_{B(i,j)} = \begin{cases} (0, 0) & \text{si } j \leq 0 \\ (\infty, \infty) & \text{si } j > 0 \text{ y } i = 0 \\ \min_{\text{lex}}(\text{OptiPago}_{B(i-1,j)}, \text{OptiPago}_{B(i-1,j-b_i)} + (b_i, 1)) & \text{en otro caso.} \end{cases}$$

Si los billetes son enteros positivos y el costo es C , hay $O(n \times C)$ estados y cada uno cuesta $O(1)$. Con memoización, el tiempo es $O(n \times C)$; sin ella, $O(2^n)$. La cota $O(n \times C)$ es pseudopolinomial.

e) Memoizamos en una matriz M inicialmente indefinida:

```
def optipago_memo(B, i, j, M):
    if j <= 0:
        return (0, 0)
    if i == 0:
        return (float("inf"), float("inf"))
    if M[i][j] is not None:
        return M[i][j]
```

```

sin = optipago_memo(B, i - 1, j, M)
monto, cantidad = optipago_memo(B, i - 1, j - B[i - 1], M)
con = (monto + B[i - 1], cantidad + 1)
M[i][j] = min(sin, con)
return M[i][j]

```

La tabla ocupa $O(n \times C)$ y la pila recursiva $O(n)$.

Las legiones del César

Problema

El general César tiene P patos y D dodos, indistinguibles entre sí. Quiere formar una fila en la que no haya más de m_P patos consecutivos ni más de m_D dodos consecutivos. Queremos contar cuántas filas válidas existen.

- Definir una función recursiva $g(t, n_P, n_D, k_P, k_D)$ usando las cantidades restantes y las rachas que terminan el prefijo actual.
- Reescribir el estado usando la última tropa y la longitud de su racha. Analizar las complejidades.
- Reducir el estado a $f(n_P, n_D, \text{ultTropa})$ agrupando las tropas en rachas. Demostrar correctitud y analizar complejidades.

Solución:

- Para decidir qué podemos agregar necesitamos saber cuántos patos y dodos quedan y cómo termina el prefijo. Definimos $g(t, n_P, n_D, k_P, k_D)$ como la cantidad de completaciones cuando quedan t lugares, n_P patos, n_D dodos, y las rachas finales tienen longitudes k_P y k_D . En estados alcanzables, $t = n_P + n_D$ y a lo sumo uno de k_P, k_D es positivo.

En cada paso podemos agregar un pato o un dodo:

$$g(t, n_P, n_D, k_P, k_D) = \begin{cases} 1 & t = 0 \\ \alpha + \beta & t > 0 \end{cases}$$

con

$$\alpha = \begin{cases} g(t-1, n_P-1, n_D, k_P+1, 0) & n_P > 0 \text{ y } k_P < m_P \\ 0 & \text{si no} \end{cases}$$

$$\beta = \begin{cases} g(t-1, n_P, n_D-1, 0, k_D+1) & n_D > 0 \text{ y } k_D < m_D \\ 0 & \text{si no} \end{cases}$$

La llamada inicial es $g(P + D, P, D, 0, 0)$.

- Como $t = n_P + n_D$, podemos eliminarlo. Además, solo una racha puede terminar el prefijo, por lo que podemos guardar una longitud k y el tipo $u \in \{P, D, \emptyset\}$ de la última tropa. Definimos $h(n_P, n_D, k, u)$.

En lugar de escribir todas las condiciones dentro de h , separamos el aporte de cada decisión. Si intentamos agregar un pato, definimos

$$\text{ponerPato}(n_P, n_D, k, u) = \begin{cases} h(n_P - 1, n_D, k + 1, P) & n_P > 0 \wedge u = P \wedge k < m_P \\ h(n_P - 1, n_D, 1, P) & n_P > 0 \wedge u \neq P \\ 0 & \text{si no.} \end{cases}$$

La primera alternativa continúa la racha de patos; la segunda comienza una nueva racha de longitud 1. Si no queda ningún pato o la racha ya alcanzó m_P , esta decisión aporta 0.

Simétricamente, el aporte de agregar un dodo es

$$\text{ponerDodo}(n_P, n_D, k, u) = \begin{cases} h(n_P, n_D - 1, k + 1, D) & n_D > 0 \wedge u = D \wedge k < m_D \\ h(n_P, n_D - 1, 1, D) & n_D > 0 \wedge u \neq D \\ 0 & \text{si no.} \end{cases}$$

Entonces la recurrencia queda

$$h(n_P, n_D, k, u) = \begin{cases} 1 & n_P = 0 \text{ y } n_D = 0 \\ \text{ponerPato}(n_P, n_D, k, u) + \text{ponerDodo}(n_P, n_D, k, u) & \text{si no.} \end{cases}$$

La llamada inicial es $h(P, D, 0, \emptyset)$. La primera versión tiene $O((P + 1)(D + 1)(m_P + 1)(m_D + 1))$ estados y la segunda $O((P + 1)(D + 1)(\max(m_P, m_D) + 1))$ estados. Con memoización, cada estado cuesta $O(1)$.

- c) También podemos interpretar los contadores como capacidades restantes: al iniciar una racha de patos la capacidad es $\min(n_P, m_P)$, y al iniciar una de dodos es $\min(n_D, m_D)$. Al agregar una tropa se consume una unidad de la capacidad correspondiente y se reinicia la capacidad del otro tipo. Esta representación no debe confundirse con la longitud actual de la racha.
- d) Finalmente decidimos una racha completa por llamada. Definimos $f(n_P, n_D, u)$ como la cantidad de continuaciones válidas cuando quedan n_P patos y n_D dodos y la última racha fue del tipo $u \in \{P, D\}$. La próxima racha debe ser del tipo opuesto:

$$f(n_P, n_D, u) = \begin{cases} 1 & n_P = 0 \text{ y } n_D = 0 \\ \sum_{i=1}^{\min(n_D, m_D)} f(n_P, n_D - i, D) & u = P \\ \sum_{i=1}^{\min(n_P, m_P)} f(n_P - i, n_D, P) & u = D. \end{cases}$$

Las sumas vacías valen 0. La respuesta es 1 si $P = D = 0$; en otro caso, es $f(P, D, P) + f(P, D, D)$, separando según el tipo de la primera racha.

Correctitud. Probamos por inducción en $n_P + n_D$. Si no quedan tropas, hay una única continuación vacía y la función devuelve 1. Supongamos ahora que quedan tropas y que la afirmación vale para todos los estados con menos tropas restantes.

Si $u = P$, la próxima racha debe ser de dodos y su longitud i está entre 1 y $\min(n_D, m_D)$. Fijada i , el estado restante es $(n_P, n_D - i, D)$; por hipótesis inductiva, la llamada cuenta correctamente sus continuaciones. Cada fila determina una única longitud para su primera racha de dodos, por lo que las alternativas son disjuntas y cubren todas las posibilidades. El caso $u = D$ es análogo: se prueban las longitudes 1 a $\min(n_P, m_P)$ y se llama a $f(n_P - i, n_D, P)$.

- e) En la versión por rachas hay a lo sumo $2(P + 1)(D + 1)$ estados. Si $R = \max(\min(P, m_P), \min(D, m_D))$, cada estado prueba a lo sumo R longitudes, así que el tiempo con memoización es $O((P + 1)(D + 1)(R + 1))$. La tabla ocupa $O((P + 1)(D + 1))$ y la pila recursiva $O(P + D)$ adicional.

Bibliografia

- [1] R. Bellman, “The theory of dynamic programming,” *Bulletin of the American Mathematical Society*, vol. 60, no. 6, pp. 503–515, 1954, doi: [10.1090/s0002-9904-1954-09848-8](https://doi.org/10.1090/s0002-9904-1954-09848-8).